EXPERIMENT NO. 6

IMPLEMENTATION AND COMPARATIVE ANALYSIS OF IMAGE SEGMENTATION

TECHNIQUES USING PYTHON AND OPENCV

Name : Akshat Chauhan

UID : CU24250136

Course : B.Tech CSE

Section : CSE "C"

Roll No. : 05

AIM

To implement and evaluate various image segmentation techniques

for partitioning digital images into meaningful regions, thereby

facilitating object localization and scene understanding.

STEP 1: IMPORT REQUIRED LIBRARIES

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from tkinter import Tk
from tkinter.filedialog import askopenfilename
from sklearn.cluster import KMeans
```

STEP 2: UPLOAD IMAGE FROM COMPUTER

```
from google.colab import files
import io

print("Please upload an image file (e.g., .jpg, .png, .bmp, .tif).")
uploaded = files.upload()

if not uploaded:
    print("No file uploaded.")
    exit()

image_filename = next(iter(uploaded)) # Get the name of the first uploaded file
image_path = io.BytesIO(uploaded[image_filename])

# Load color image
# For images uploaded via files.upload(), cv2.imread needs a file path or a numpy array.
# We will use cv2.imdecode to read the image from bytes.
image = cv2.imdecode(np.frombuffer(uploaded[image_filename], np.uint8), cv2.IMREAD_COLOR)

if image is None:
    print("Error: Unable to load the selected image.")
    exit()

print("Image loaded successfully.")
print("Selected Image:", image_filename)
```

```
Please upload an image file (e.g., .jpg, .png, .bmp, .tif).
Choose Files No file chosen    Uploadwidgetisonlyavailablewhenthecellhasbeenexecuted in the current browser session. Please rerun this cell to enable.
Saving Screenshot 2025-03-08 134557.png to Screenshot 2025-03-08 134557.png
Image loaded successfully.
Selected Image: Screenshot 2025-03-08 134557.png
```

STEP 3: CONVERT IMAGE TO GRAYSCALE AND APPLY GAUSSIAN BLUR

```
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

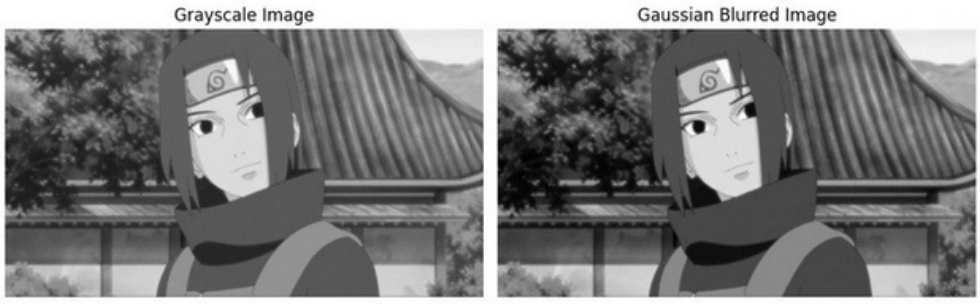
blurred = cv2.GaussianBlur(
    gray,
    (5, 5),
    0
)

plt.figure(figsize=(10, 5))

plt.subplot(1, 2, 1)
plt.imshow(gray, cmap="gray")
plt.title("Grayscale Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(blurred, cmap="gray")
plt.title("Gaussian Blurred Image")
plt.axis("off")

plt.tight_layout()
plt.show()
```



STEP 4: GLOBAL THRESHOLDING

```
_, global_threshold = cv2.threshold(
    blurred,
    127,
    255,
    cv2.THRESH_BINARY
)

plt.figure(figsize=(6, 5))
plt.imshow(global_threshold, cmap="gray")
plt.title("Global Thresholding")
plt.axis("off")
plt.show()
```



STEP 5: OTSU'S THRESHOLDING

```
otsu_value, otsu_threshold = cv2.threshold(
    blurred,
    0,
    255,
    cv2.THRESH_BINARY + cv2.THRESH_OTSU
)

print("Otsu threshold value:", round(otsu_value, 2))

plt.figure(figsize=(6, 5))
plt.imshow(otsu_threshold, cmap="gray")
plt.title("Otsu's Thresholding")
plt.axis("off")
plt.show()
```

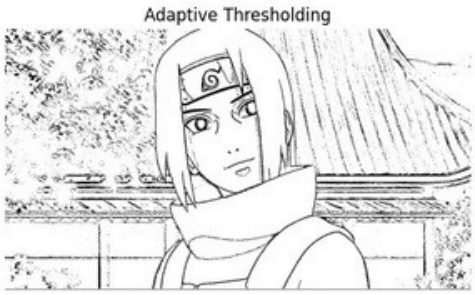
```
Otsu threshold value: 131.0
```



STEP 6: ADAPTIVE THRESHOLDING

```
adaptive_threshold = cv2.adaptiveThreshold(
    blurred,
    255,
    cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY,
    11,
    2
)

plt.figure(figsize=(6, 5))
plt.imshow(adaptive_threshold, cmap="gray")
plt.title("Adaptive Thresholding")
plt.axis("off")
plt.show()
```



STEP 7: WATERSHED SEGMENTATION

```
_, binary = cv2.threshold(
    blurred,
    0,
    255,
    cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU
)

# Remove noise using morphological opening
kernel = np.ones((3, 3), np.uint8)

opening = cv2.morphologyEx(
    binary,
    cv2.MORPH_OPEN,
    kernel,
    iterations=2
)

# Find sure background
sure_bg = cv2.dilate(
    opening,
    kernel,
    iterations=3
)

# Find distance from background
dist_transform = cv2.distanceTransform(
    opening,
    cv2.DIST_L2,
    5
)

# Find sure foreground
_, sure_fg = cv2.threshold(
    dist_transform,
    0.7 * dist_transform.max(),
    255,
    0
)

sure_fg = np.uint8(sure_fg)

# Find unknown region
unknown = cv2.subtract(
    sure_bg,
    sure_fg
)

# Create markers
_, markers = cv2.connectedComponents(sure_fg)

markers = markers + 1
markers[unknown == 255] = 0

# Apply Watershed
watershed_image = image.copy()

markers = cv2.watershed(
 watershed_image,
 markers
)

watershed_image[markers == -1] = [255, 0, 0]

watershed_rgb = cv2.cvtColor(
    watershed_image,
    cv2.COLOR_BGR2RGB
)

plt.figure(figsize=(8, 6))
plt.imshow(watershed_rgb)
plt.title("Watershed Segmentation")
plt.axis("off")
plt.show()
```



STEP 8: K-MEANS COLOR IMAGE SEGMENTATION

```
small_image = cv2.resize(
    image_rgb,
    (300, 300)
)

pixels = small_image.reshape(
    (-1, 3)
)

pixels = np.float32(pixels)

# Apply K-Means
k = 3

kmeans = KMeans(
    n_clusters=k,
    random_state=42,
    n_init=10
)

labels = kmeans.fit_predict(pixels)

centers = np.uint8(
    kmeans.cluster_centers_
)

segmented_pixels = centers[labels]

segmented_image = segmented_pixels.reshape(
    small_image.shape
)

plt.figure(figsize=(6, 5))
plt.imshow(segmented_image)
plt.title("K-Means Segmentation")
plt.axis("off")
plt.show()
```



STEP 9: COMPARISON OF ALL SEGMENTATION TECHNIQUES

```
plt.figure(figsize=(15, 10))

plt.subplot(2, 3, 1)
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis("off")

plt.subplot(2, 3, 2)
plt.imshow(global_threshold, cmap="gray")
plt.title("Global Thresholding")
plt.axis("off")
```

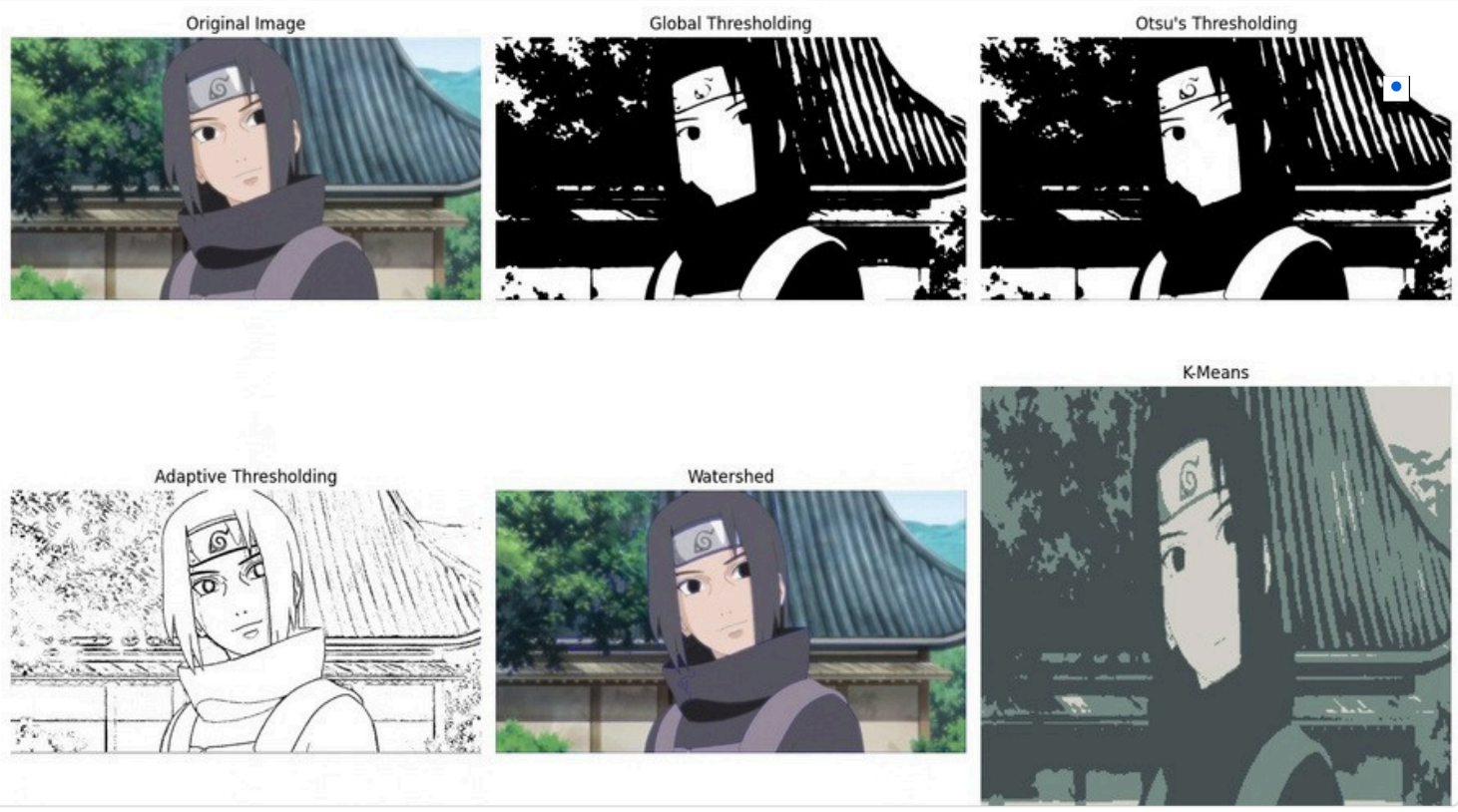
```
plt.subplot(2, 3, 3)
plt.imshow(otsu_threshold, cmap="gray")
plt.title("Otsu's Thresholding")
plt.axis("off")

plt.subplot(2, 3, 4)
plt.imshow(adaptive_threshold, cmap="gray")
plt.title("Adaptive Thresholding")
plt.axis("off")

plt.subplot(2, 3, 5)
plt.imshow(watershed_rgb)
plt.title("Watershed")
plt.axis("off")

plt.subplot(2, 3, 6)
plt.imshow(segmented_image)
plt.title("K-Means")
plt.axis("off")

plt.tight_layout()
plt.show()
```



STEP 10: DISPLAY IMAGE HISTOGRAM

```
plt.figure(figsize=(8, 5))

plt.hist(
    gray.ravel(),
    bins=256,
    range=(0, 256)
)

plt.title("Grayscale Image Histogram")
plt.xlabel("Pixel Intensity")
plt.ylabel("Frequency")
plt.show()
```

Q1. What is image segmentation? Why is it considered a fundamental step in computer vision?

Image segmentation is the process of dividing an image into meaningful regions or objects based on pixel intensity, color, texture, or boundaries.

It is fundamental because it separates regions of interest from the background and makes later computer vision tasks easier.

Q2. Differentiate between Image Segmentation and Image Classification.

Image Segmentation divides an image into different regions or individual objects at pixel level.

Image Classification assigns a complete image to a particular category or class.

Example: Segmentation identifies the pixels belonging to a car. Classification identifies the image as a car image.

Q3. Explain the working principle of Global Thresholding, Otsu's Thresholding, and Adaptive Thresholding.

Global Thresholding uses one fixed threshold value for the complete image to separate foreground and background.

Otsu's Thresholding automatically calculates an optimal threshold from the image histogram.

Adaptive Thresholding calculates different threshold values for different local regions of the image, making it useful for images with varying illumination.

Q4. What is the Watershed Algorithm? Why is it useful for separating overlapping objects?

Watershed is a region-based segmentation algorithm that treats an image like a topographic surface.

It is useful for separating touching or overlapping objects because marker regions can be used to identify individual objects.

Q5. How is K-Means Clustering applied to image segmentation?

Image pixels are treated as data points.

K-Means groups pixels into K clusters according to their color or intensity.

Each pixel is then replaced by the color of its cluster centroid, producing the segmented image.

Q6. Compare threshold-based segmentation and clustering-based segmentation techniques.

Threshold-based segmentation separates pixels using threshold values. It is simple and computationally efficient.

Clustering-based segmentation groups pixels according to similarity in color or intensity and can handle multiple regions more flexibly.

Thresholding is generally faster, while clustering can provide better segmentation for complex color images.

Q7. What challenges are encountered while segmenting images with complex backgrounds or varying illumination?

Major challenges include:

Uneven illumination Noise Similar foreground and background colors Complex backgrounds Object overlap Shadows and reflections

Adaptive thresholding, preprocessing, Watershed, and clustering can help overcome some of these challenges.

Q8. Mention five real-world applications where image segmentation plays a critical role.

Medical image analysis Autonomous vehicle navigation Satellite image analysis Industrial inspection Object detection and recognition

Q9. How does image segmentation improve the performance of object detection and image recognition systems?

Segmentation isolates important objects or regions from the background.

This reduces irrelevant information and allows detection and recognition systems to focus on the region of interest, improving accuracy and efficiency.

Q10. Compare traditional image segmentation techniques with deep learning-based segmentation methods such as U-Net and Mask R-CNN.

Traditional techniques such as thresholding, Watershed, and K-Means are generally simple, fast, and do not require large training datasets.

Deep learning methods such as U-Net and Mask R-CNN can learn complex features and usually provide better segmentation for difficult real-world images.

However, deep learning methods require large labelled datasets, greater computational resources, and more training time.